

Major design choices

- Created helper functions for `Imply` and `Iff` behaviour to help evaluation code remain readable.
- In `findVarIds` I decide to use the `nub` function (part of the `Data.List` module) to only a single copy of each variable
- `genVarAsgns` generates all assignments by recursion: the base case (empty variable list) returns `[emptyMap]`. As the recursion bubbles back up it branches take all assignments for `xs` (elems) and extend each map in two ways by `interesting var = True` and `var = False`. Hence each step doubles the number of variable assignments
- Since the custom type “`Prop`” derives from `Read` I was able to transform an input string to a `Prop` type by utilizing the `read` function.

Testing

I created a bash test script to run a set of 25 formulas

Corner cases considered:

- Formulas with constants
- Deeply nested formulas
- Contradictions such as `p AND (NOT p)`